UNIVERSIDAD NACIONAL DE LOJA

Carrera de Computación

Resumen de Sistemas Digitales

RICHARD ARIEL CHAMBA SANTIN

Unidad 1 · Microcontroladores para sistemas embebidos

8 CLASES **18 PÁGINAS DE APUNTES ORIGINALES**

CONTENIDO

01	Microcontroladores y arquitectura	02
02	CPU, buses y reloj	03
03	Memorias, bits y ensamblador	05
04	Entradas y salidas en AVR	10
05	Del AVR al ESP32	11
06	Arranque, tipos y pulsadores	13
07	Temporizadores de hardware	15
08	Interrupciones externas	18

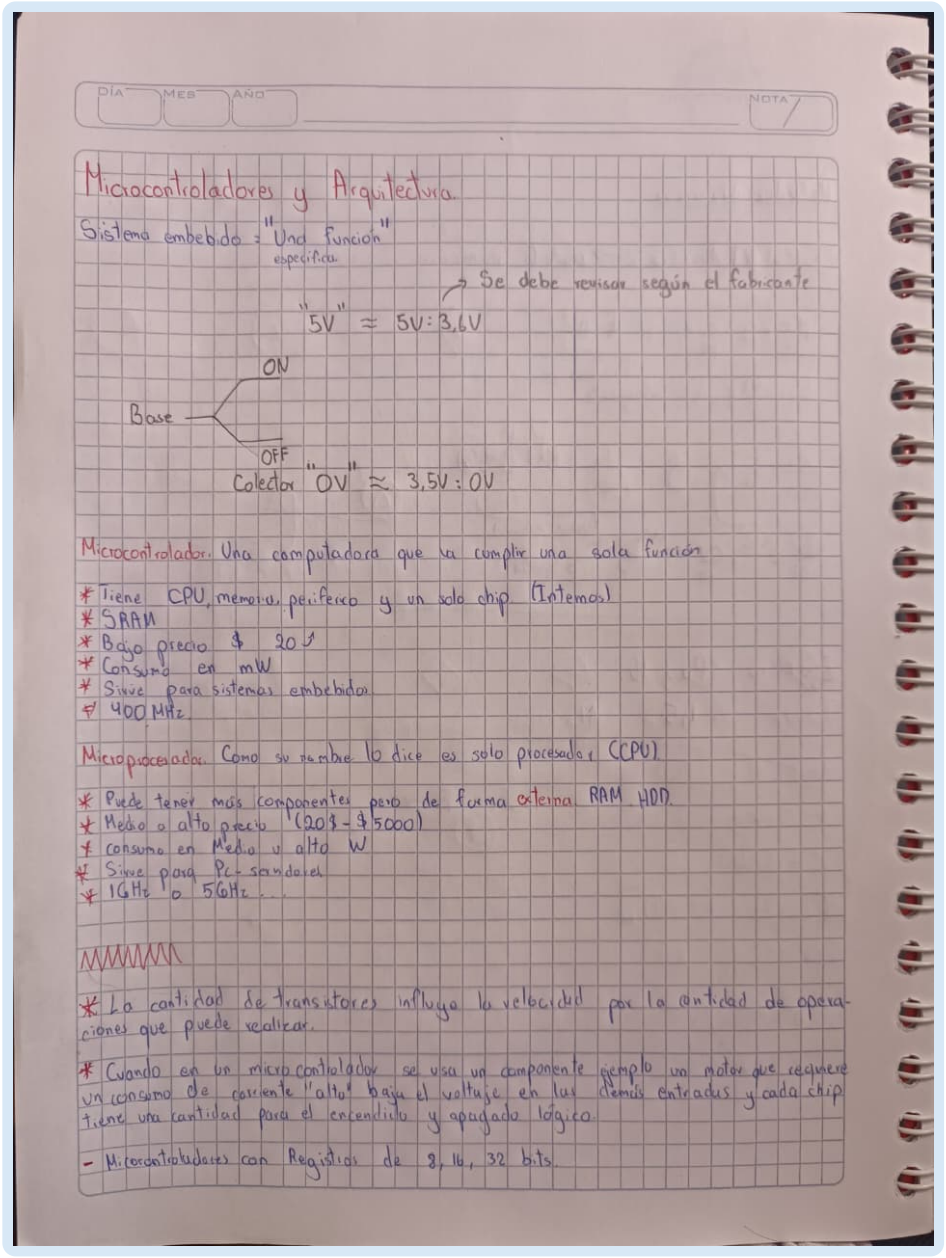
Mis páginas manuscritas, reunidas en una composición azul con títulos, ideas clave y precisiones de lectura. Las capturas conservan el contenido original; las aclaraciones se presentan fuera de ellas.

Fórmulas: p. 20 · Ejercicios: p. 21 · Uso de IA: p. 22

Material de clase contrastado con las seis presentaciones del curso.

Microcontroladores y arquitectura

CPU · memoria · sistemas embebidos



IDEA CENTRAL

Un microcontrolador integra CPU, memoria y periféricos en un chip.

ACLARACIÓN

5 V y 3,3 V son referencias de alimentación y lógica. Los umbrales exactos dependen del fabricante.

PARA RECORDAR

Un motor necesita una etapa de potencia adecuada: no se alimenta desde un GPIO.

PARA ELEGIR EL MCU

¿Cuánta memoria necesito? ¿Cuántos pines? ¿Hace falta Wi-Fi? Elegir según la tarea, no solo por la frecuencia.

EN UNA IDEA

CPU + memoria + periféricos en un chip. Precio, frecuencia, consumo y umbrales de voltaje dependen del dispositivo; las cifras manuscritas son referencias, no reglas universales.

Clase 1.jpeg · Diapositivas S01, sesión 1, pp. 4–8 y 18.

Bloques del microcontrolador

Tres buses, tres preguntas



TRES PREGUNTAS

Direcciones: ¿dónde?
Datos: ¿qué contenido?
Control: ¿leer o escribir?

DENTRO DE LA CPU

ALU: calcula y compara.
Registros: guardan datos de trabajo con acceso muy rápido.

ACLARACIÓN

FPGA es hardware lógico configurable. No es un lenguaje ensamblador.

CAPACIDAD

n bits permiten 2^n combinaciones. Con 10 bits de dirección se distinguen 1024 posiciones.

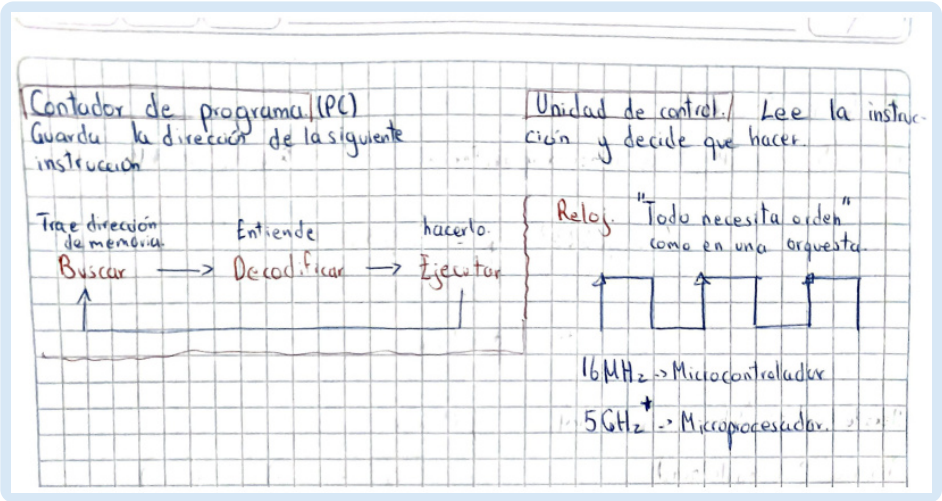
EN UNA IDEA

Direcciones: dónde. Datos: qué contenido. Control: qué operación. FPGA es hardware configurable; no es un lenguaje ensamblador.

Clase 2 (1).pdf · p. 1 · S01, sesión 1, pp. 5–9 y 13.

El ciclo de la CPU y el reloj

Buscar → decodificar → ejecutar



Buscar → **Decodificar** → **Ejecutar**

La secuencia se repite al ritmo del reloj.

PC

El contador de programa guarda la dirección de la siguiente instrucción.

EL CICLO

Buscar → decodificar → ejecutar. La CPU repite la secuencia al ritmo del reloj.

EJEMPLO

16 MHz = 16 millones de ciclos por segundo. Un ciclo dura 62,5 ns; una instrucción puede necesitar varios.

FRECUENCIA

f = 1 / T; T = 1 / f.
f en Hz y T en segundos.
1 MHz = 1 000 000 Hz.

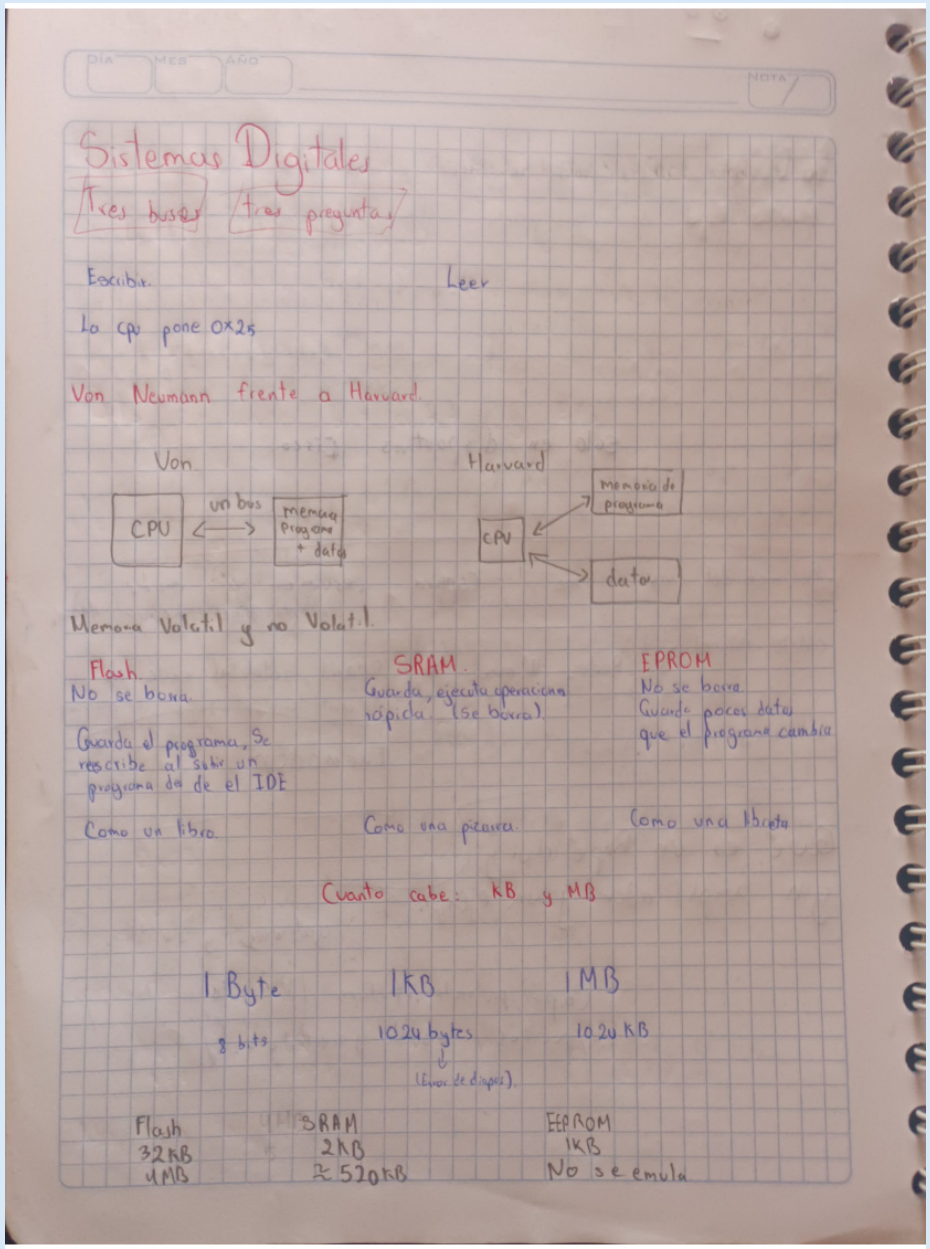
EN UNA IDEA

La CPU busca, decodifica y ejecuta. El reloj de 16 MHz del Uno tiene ciclos de 62,5 ns; los 5 GHz del apunte son otro ejemplo, no una frecuencia universal.

Clase 2 (1).pdf · p. 2 · S01, sesión 1, pp. 7–8 y 18.

Arquitecturas y memorias

Von Neumann · Harvard · Flash · SRAM · EEPROM



NO SE BORRAN

Flash: programa.
EEPROM: datos que deben permanecer después de apagar.

SÍ SE BORRA

SRAM: variables durante la ejecución. La CPU ejecuta; la memoria almacena.

CORRECCIÓN

Donde el apunte dice EPROM, la descripción corresponde a EEPROM. No son el mismo tipo de memoria.

UNIDADES

1 byte = 8 bits.
En la convención de clase: 1 KB = 1024 bytes; 1 MB = 1024 KB.

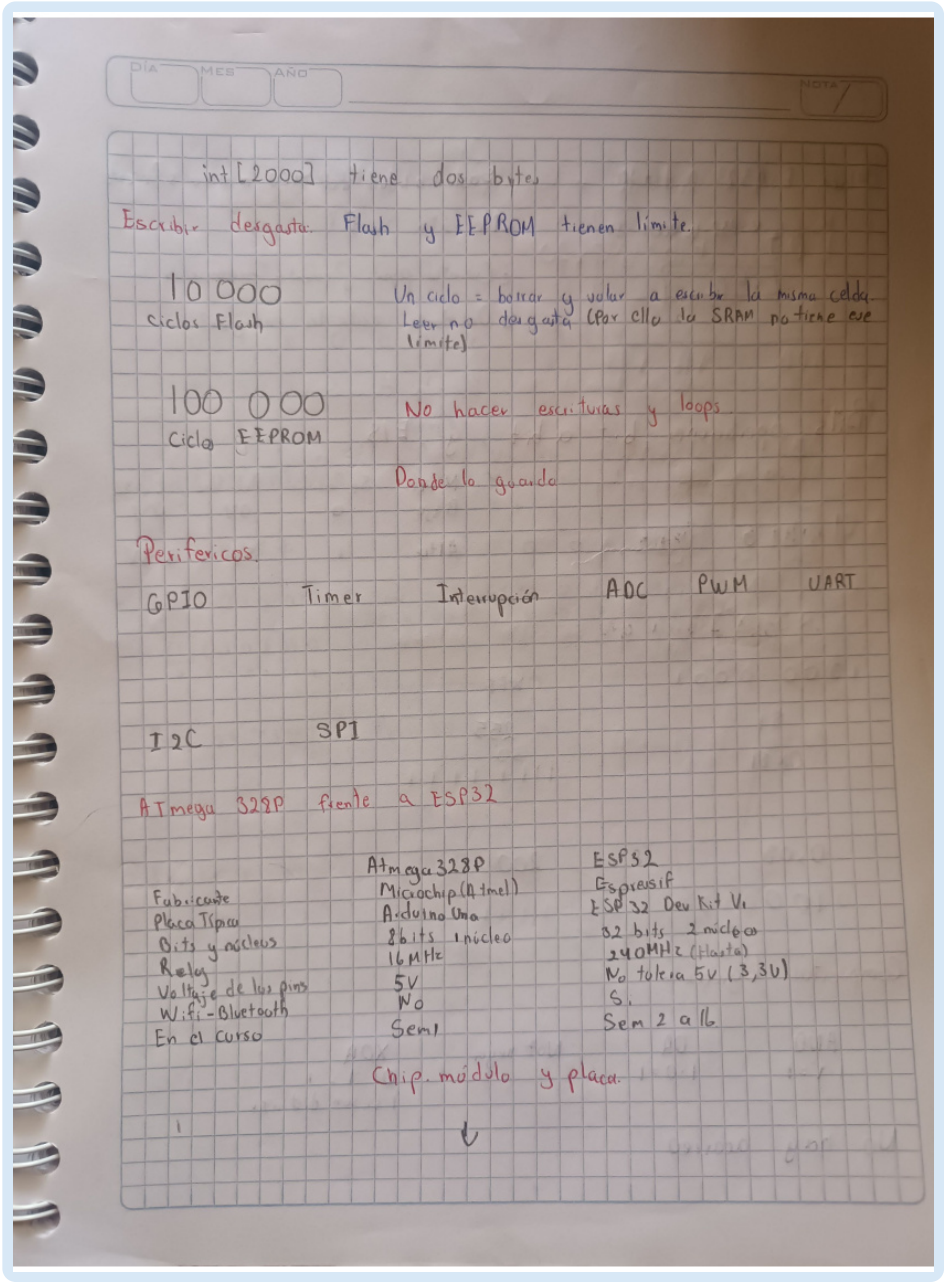
EN UNA IDEA

Flash conserva el programa; SRAM guarda variables mientras hay energía; EEPROM conserva datos. En esta página, donde dice EPROM corresponde EEPROM.

Sem3.pdf · p. 1 · S01, sesión 1, pp. 9–14.

Capacidad, periféricos y plataformas

ATmega328P frente a ESP32



EL ARREGLO

En el AVR del Uno, un int ocupa 2 bytes. int[2000] necesita 4000 bytes: no cabe en 2 KB de SRAM.

DOS PLATAFORMAS

Uno: 8 bits y 16 MHz. ESP32 clásico de clase: 32 bits y hasta 240 MHz.

ESCRITURA

Los ciclos indicados son referencias del ATmega328P. Evitar guardar repetidamente el mismo dato en EEPROM.

TAMAÑO TOTAL

bytes del arreglo = elementos × bytes de cada elemento.
2000 × 2 = 4000 bytes.

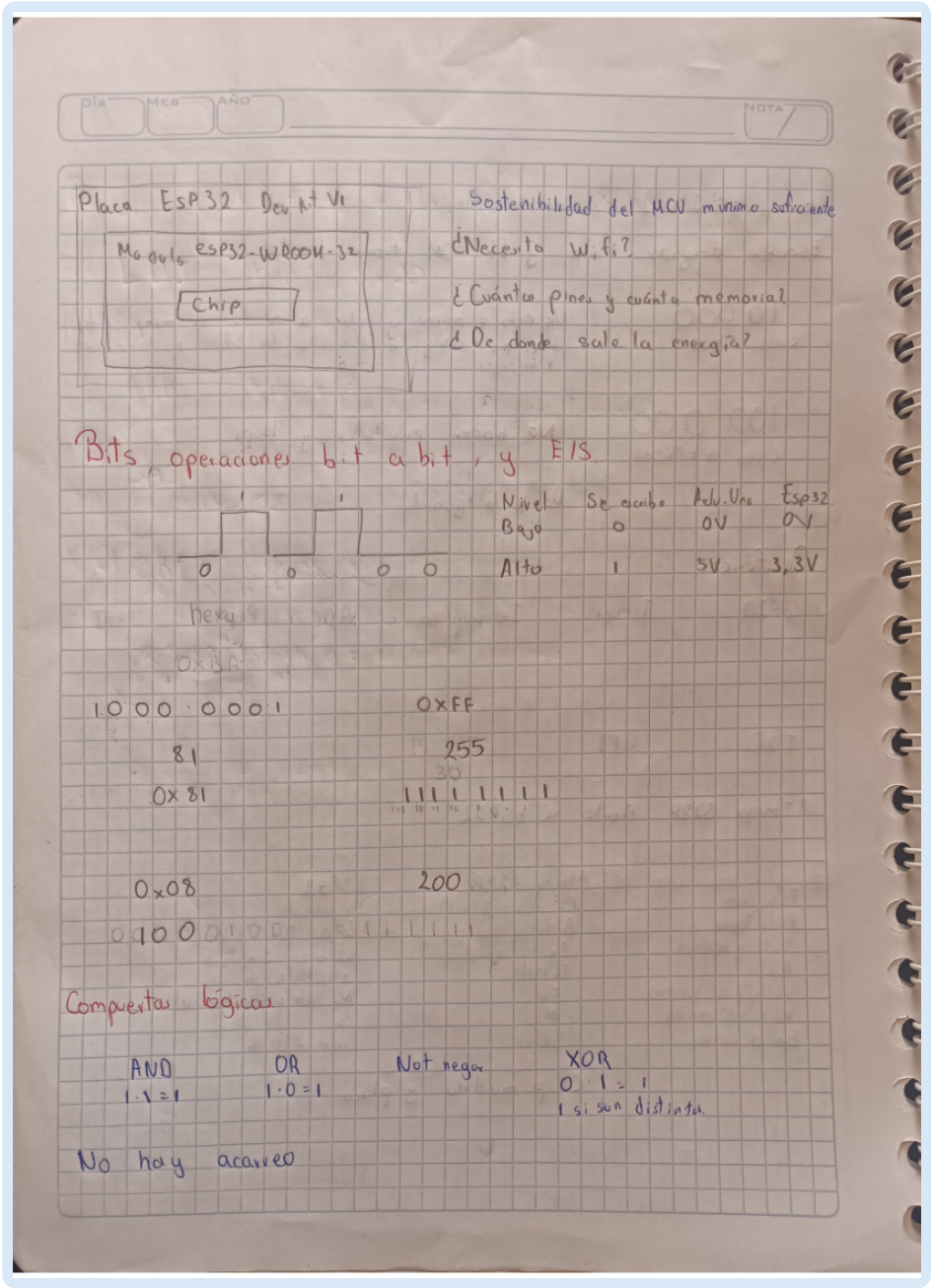
EN UNA IDEA

En AVR, cada int ocupa 2 bytes: int[2000] necesita 4000 bytes y no cabe en 2 KB de SRAM. En el ESP32 de clase, int suele ocupar 4 bytes.

Sem3.pdf · p. 2 · S01, sesión 1, pp. 14–19; S2, sesión 2, p. 6.

Bits y operaciones lógicas

Binario · hexadecimal · compuertas



CONVERSIONES

0x81 = 129
0xFF = 255
0x08 = 8
200 = 0xC8

PESO DE LOS BITS

De derecha a izquierda: 1, 2, 4, 8, 16, 32, 64 y 128. Se suman los pesos que están en 1.

LÓGICA

AND: ambos en 1.
OR: al menos uno en 1.
XOR: distintos.
NOT: invierte.

RANGO SIN SIGNO

Con n bits: de 0 a $2^n - 1$.
Para 8 bits: 0 a 255.
Un dígito hexadecimal representa 4 bits.

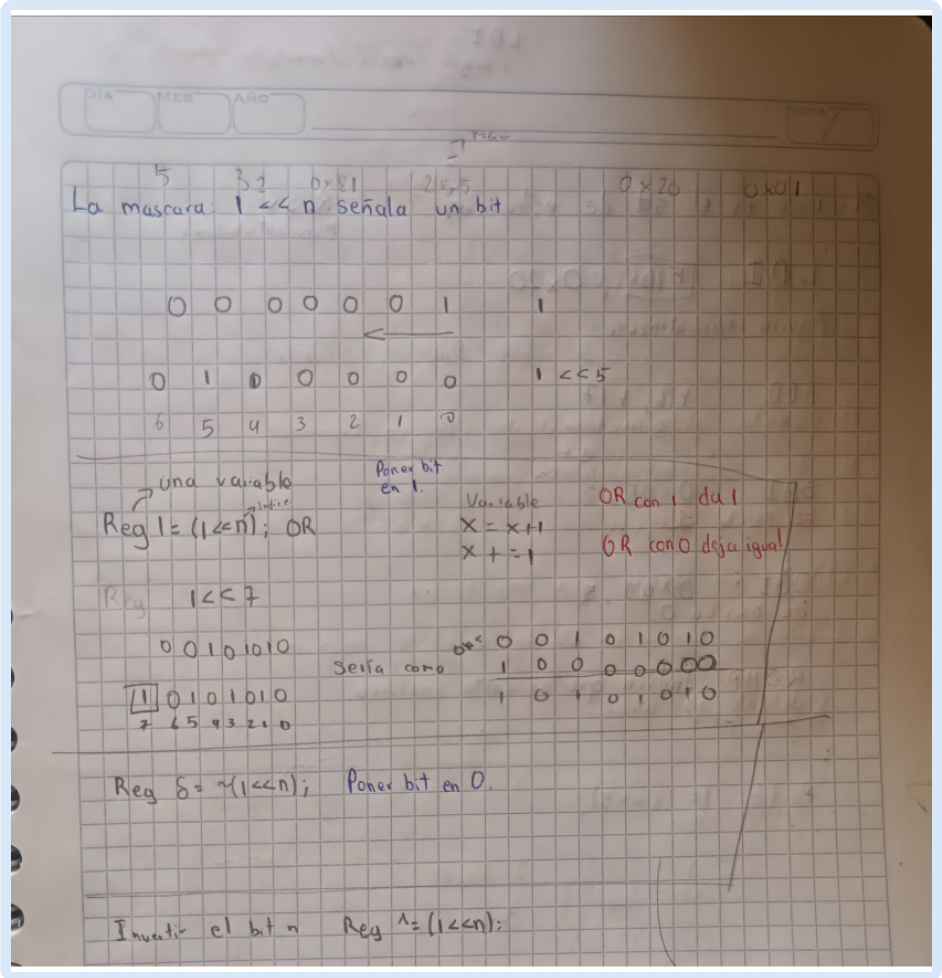
EN UNA IDEA

Conversiones completas: 0x81 = 129 = 1000 0001; 0xFF = 255; 0x08 = 0000 1000; 200 = 0xC8 = 1100 1000.

Sem3.pdf · p. 3 · S01, sesión 1, pp. 19–20; S01, sesión 2, pp. 5–10.

Máscaras: modificar un solo bit

Poner en 1 · borrar · invertir · consultar



MÁSCARA

1 << 5 = 0010 0000 = 0x20. Señala el bit 5; el primer bit se numera 0.

TRES RECETAS

|= pone en 1.
&= ~ pone en 0.
^= invierte.
Los otros bits se conservan.

EN ESP32

Para máscaras de 32 bits se usa 1UL << n. El desplazamiento debe estar dentro del rango del registro.

CONSULTAR UN BIT

(REG & (1 << n)) != 0
Es verdadero si el bit n está en 1. No modifica el registro.

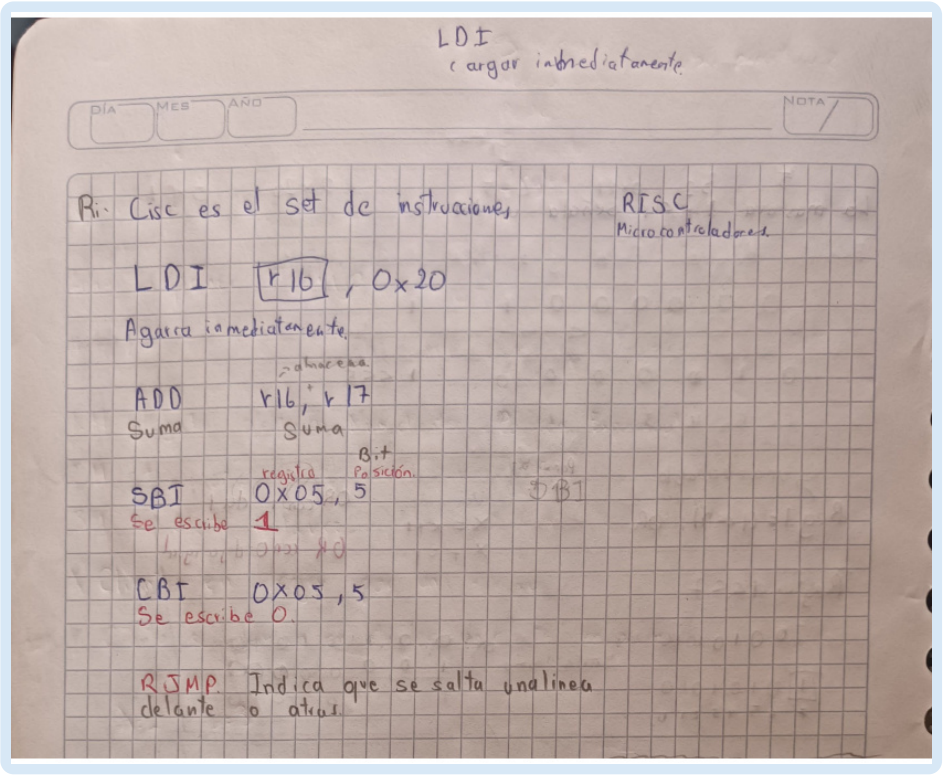
EN UNA IDEA

1 << n crea la máscara. |= pone en 1; &= ~ pone en 0; ^= invierte. Para máscaras de 32 bits en ESP32 se utiliza 1UL << n.

Sem3.pdf · p. 4 · S01, sesión 2, pp. 11–13; S2, sesión 2, p. 10.

Instrucciones del AVR

Del código a las operaciones del chip



INMEDIATO

LDI r16, 0x20 carga el valor 0x20 en r16. “Inmediato” significa que el valor forma parte de la instrucción.

OPERACIONES

ADD suma.
SBI pone un bit en 1.
CBI lo pone en 0.
RJMP realiza un salto relativo.

DOS DIRECCIONES

PORTB es 0x05 en el espacio de E/S del AVR y 0x25 en el espacio de datos.

DEL C AL AVR

PORTB |= (1 << 5);
puede convertirse en:
SBI 0x05, 5
para el ATmega328P.

LDI r16, 0x20 → **r16 = 0010 0000** → **32 en decimal**

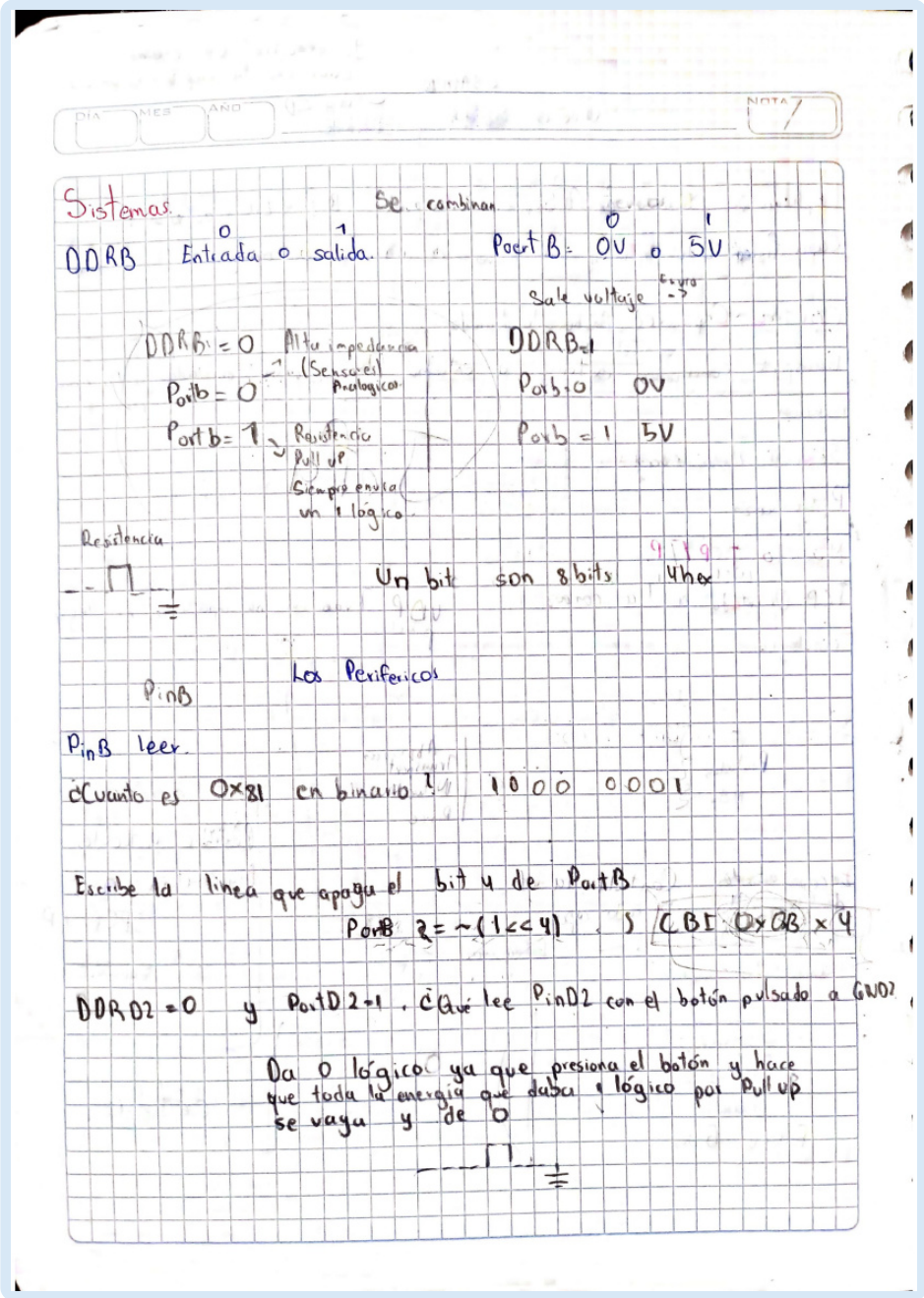
EN UNA IDEA

LDI carga un valor inmediato; ADD suma; SBI pone un bit en 1; CBI lo pone en 0; RJMP salta. PORTB: dirección de E/S 0x05, dirección de datos 0x25.

Sem3.pdf · p. 5 · S01, sesión 2, pp. 14–17.

DDRB, PORTB y PINB

Configurar · escribir · leer



LOS REGISTROS

DDR configura entrada o salida. PORT escribe el nivel o activa la pull-up. PIN lee el pin.

BOTÓN A GND

Entrada con pull-up:
suelto = 1;
presionado = 0.
La resistencia limita la corriente.

CORRECCIONES

Un byte son 8 bits.
Para apagar PB4 en ensamblador: CBI 0x05, 4.

ENTRADA FLOTANTE

DDR = 0 y PORT = 0: sin pull-up. Si no hay una señal externa definida, la lectura puede variar por ruido.

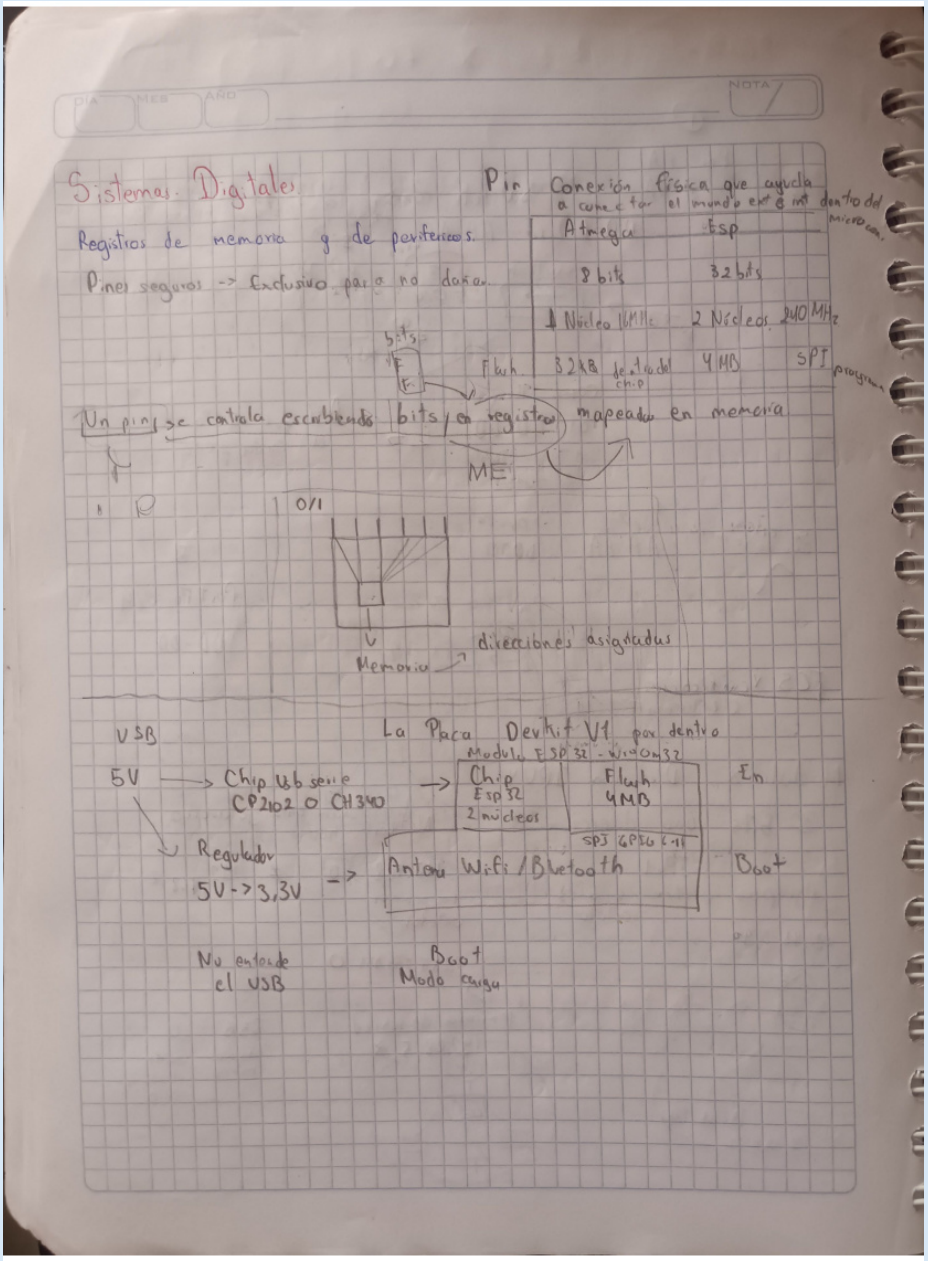
EN UNA IDEA

Correcciones: un byte son 8 bits. Para apagar PB4: PORTB &= ~(1 << 4); o CBI 0x05, 4. Con entrada pull-up y botón a GND: pulsado = 0.

calse4.pdf · p. 1 · S01, sesión 2, pp. 18–23.

Registros, pines y placa ESP32

Un pin se controla escribiendo bits



TRES NIVELES

Chip → módulo → placa. El módulo WROOM integra el ESP32, la Flash y la antena.

ALIMENTACIÓN

USB: 5 V.
Regulador: 3,3 V.
Los GPIO trabajan con lógica de 3,3 V, no de 5 V.

USB-SERIE

CP2102 o CH340 permite comunicar el computador con el ESP32 clásico de la placa.

RESISTENCIA LED

R = (3,3 V − Vf) / I.
Con LED rojo de Vf = 2 V y R = 330 Ω: I ≈ 3,94 mA.
S2, sesión 1, p. 15.

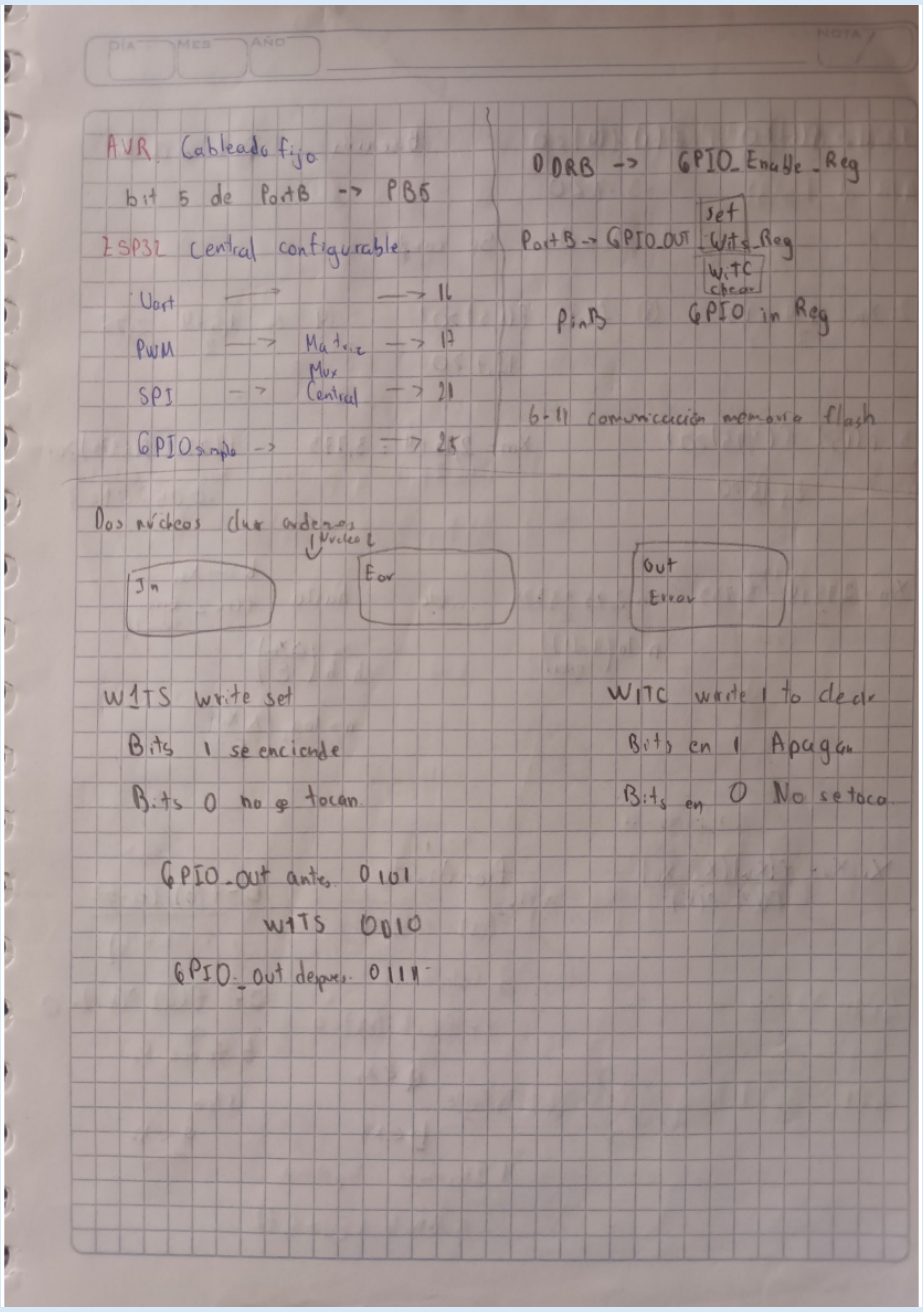
EN UNA IDEA

La placa recibe 5 V por USB y regula a 3,3 V. Los GPIO del ESP32 trabajan a 3,3 V. Chip, módulo y placa son componentes distintos.

clase 5.pdf · p. 1 · S2, sesión 1, pp. 4–8 y 13–15.

Del AVR a la matriz GPIO

W1TS · W1TC · registros de entrada y salida



W1TS

Write 1 to set. Escribir 1 activa el bit; escribir 0 no lo modifica.

W1TC

Write 1 to clear. Escribir 1 borra el bit; escribir 0 no lo modifica.

EJEMPLO

Antes: 0101
W1TS: 0010
Después: 0111
Se evita leer y reescribir todo el registro.

SIN LEER ANTES

W1TS/W1TC modifican solo los bits señalados. Una escritura evita reescribir por error otros bits a partir de una lectura anterior.

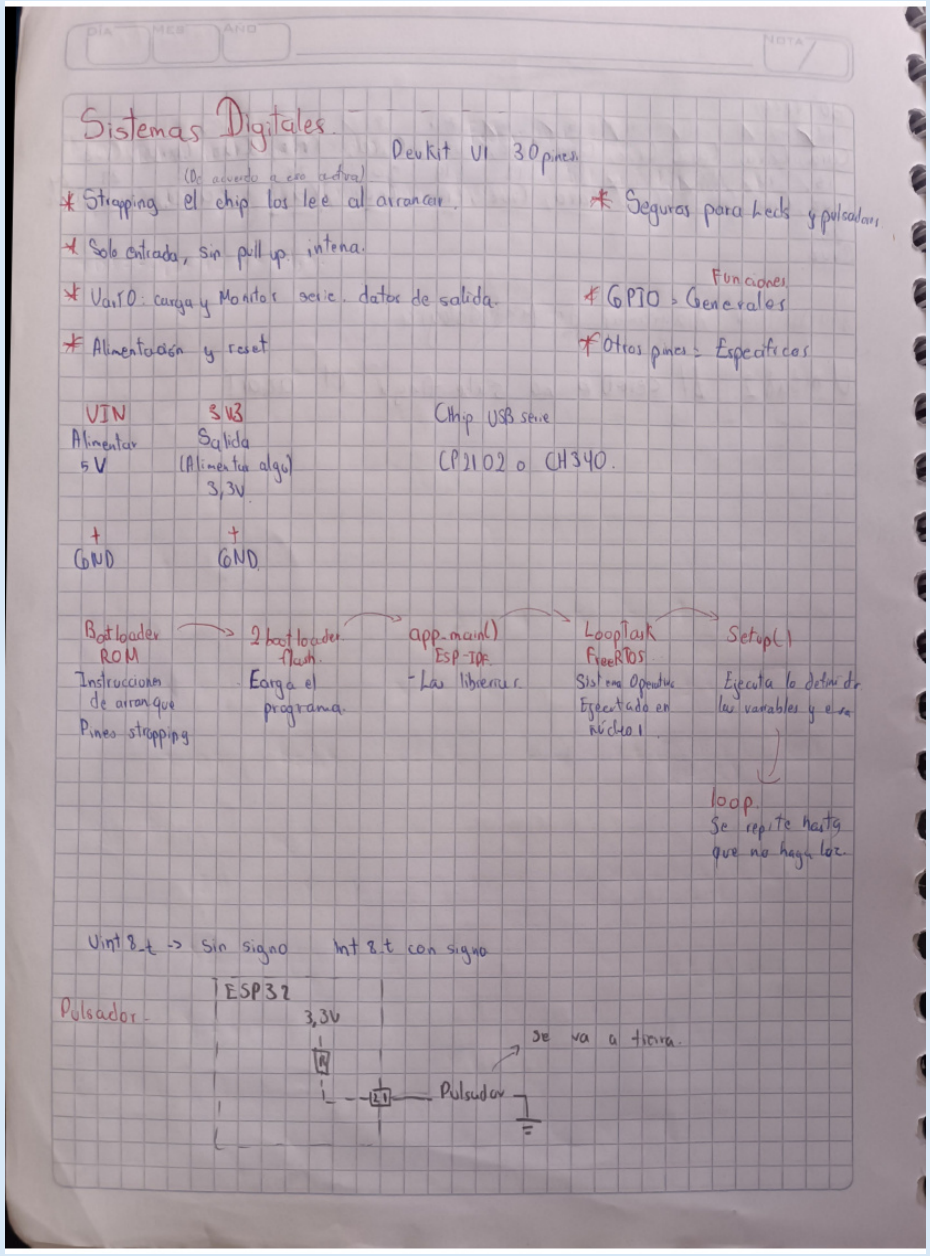
EN UNA IDEA

W1TS: un 1 activa el bit. W1TC: un 1 lo borra. En ambos, un 0 no cambia nada. Las rutas de UART, PWM y SPI del dibujo son ejemplos configurables.

clase 5.pdf · p. 2 · S2, sesión 1, pp. 5–10.

Arranque, alimentación y pulsadores

DevKit V1 · 30 pines · setup y loop



ARRANQUE

ROM → bootloader en Flash → app_main → loopTask → setup → loop.

TIPOS

uint8_t: 0 a 255.
int8_t: −128 a 127.
Ambos ocupan 8 bits; cambia su interpretación.

PULSADOR

GPIO21 con pull-up: al soltar lee 1, al presionar lee 0. GPIO34–39 no tienen pull-up interna.

CON SIGNO

En complemento a dos, n bits: -2^{n-1} a $2^{n-1}-1$.
Con 8 bits: −128 a 127.
S2, sesión 2, pp. 5–7.

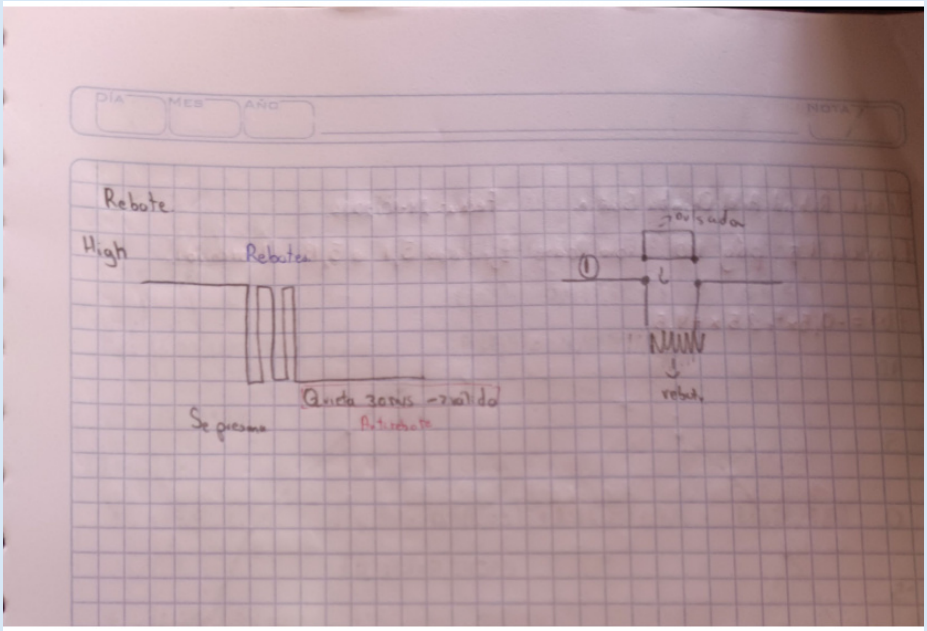
EN UNA IDEA

setup() se ejecuta una vez por arranque; loop() se repite. uint8_t: 0–255; int8_t: −128–127. GPIO34–39 solo admiten entrada y no tienen pull-up interna.

Clase6_2.pdf · p. 1 · S2, sesión 1, pp. 13 y 19–20; S2, sesión 2, pp. 7 y 20.

Rebote y antirrebote

Una pulsación física, varios cambios eléctricos





QUÉ SE VE

El contacto mecánico no cambia una sola vez: rebota y produce varios flancos rápidos.

VALIDACIÓN

El ejemplo acepta el cambio después de 30 ms de estabilidad. Ese tiempo es una elección de diseño.

UNA PULSACIÓN

Después del antirrebote, detectar HIGH → LOW evita contar en cada vuelta de loop mientras se mantiene pulsado.

TIEMPO TRANSCURRIDO

ahora − tAnterior ≥ intervalo. Para antirrebote: esperar estabilidad; no detener todo el loop con una espera larga.

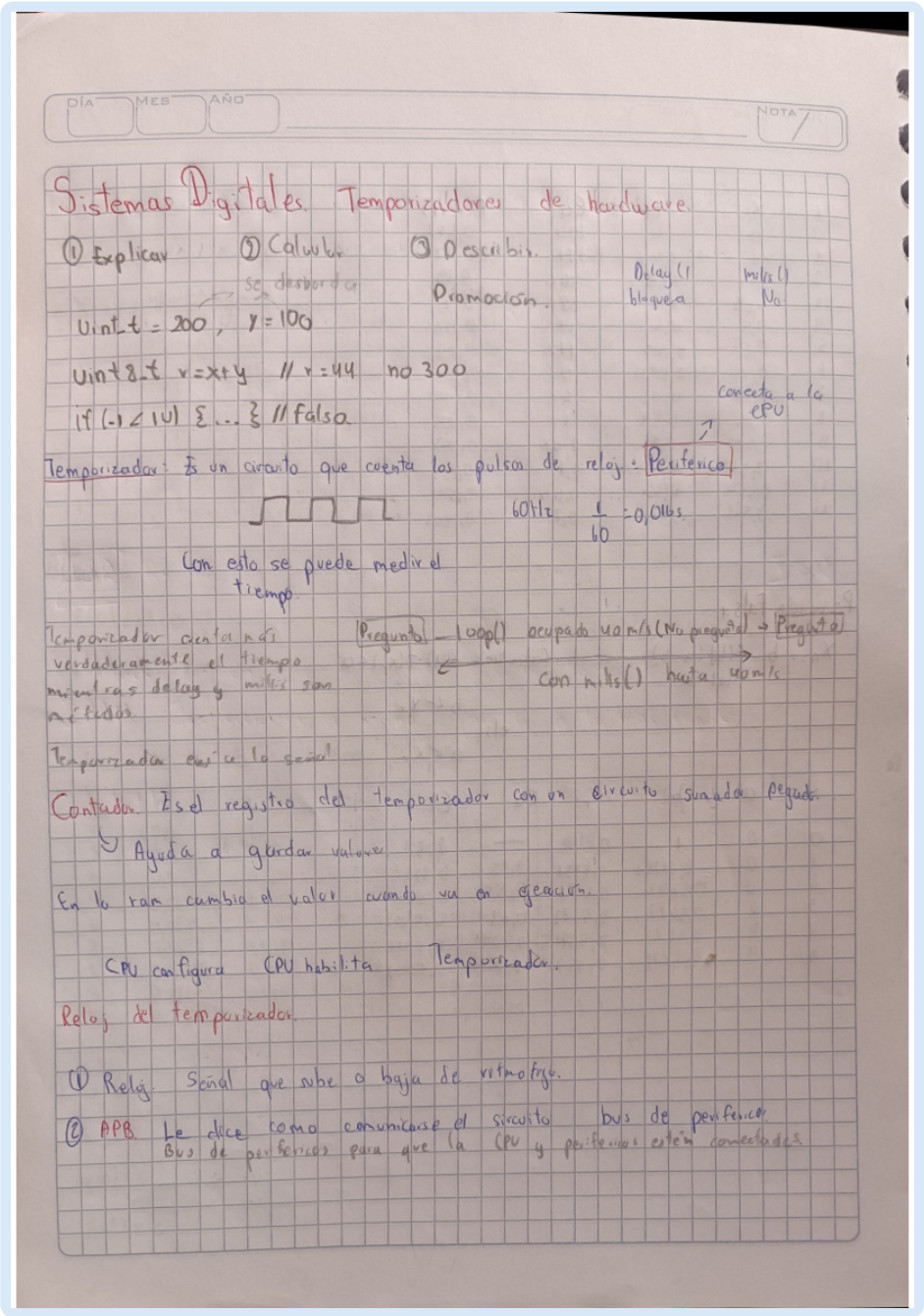
EN UNA IDEA

Una pulsación puede producir varios flancos por rebote. El ejemplo valida 30 ms de estabilidad; después se detecta el flanco para contar una sola pulsación.

Clase6_2.pdf · p. 2 · S2, sesión 2, pp. 15–23.

Temporizadores de hardware

Explicar · calcular · describir



ES HARDWARE

El contador es un registro del periférico. No es una variable de la RAM ni requiere una instrucción por cada tick.

REPASO DE TIPOS

200 + 100 = 300. Al guardar en uint8_t queda 44, porque 300 − 256 = 44.

MEDIR TIEMPO

A 60 Hz: T = 1/60 s ≈ 16,7 ms. El timer cuenta; la CPU puede ocuparse de otras tareas.

DOS FRECUENCIAS

f: ticks por segundo.
f_aviso: avisos por segundo.
No son lo mismo: un aviso puede necesitar muchos ticks.

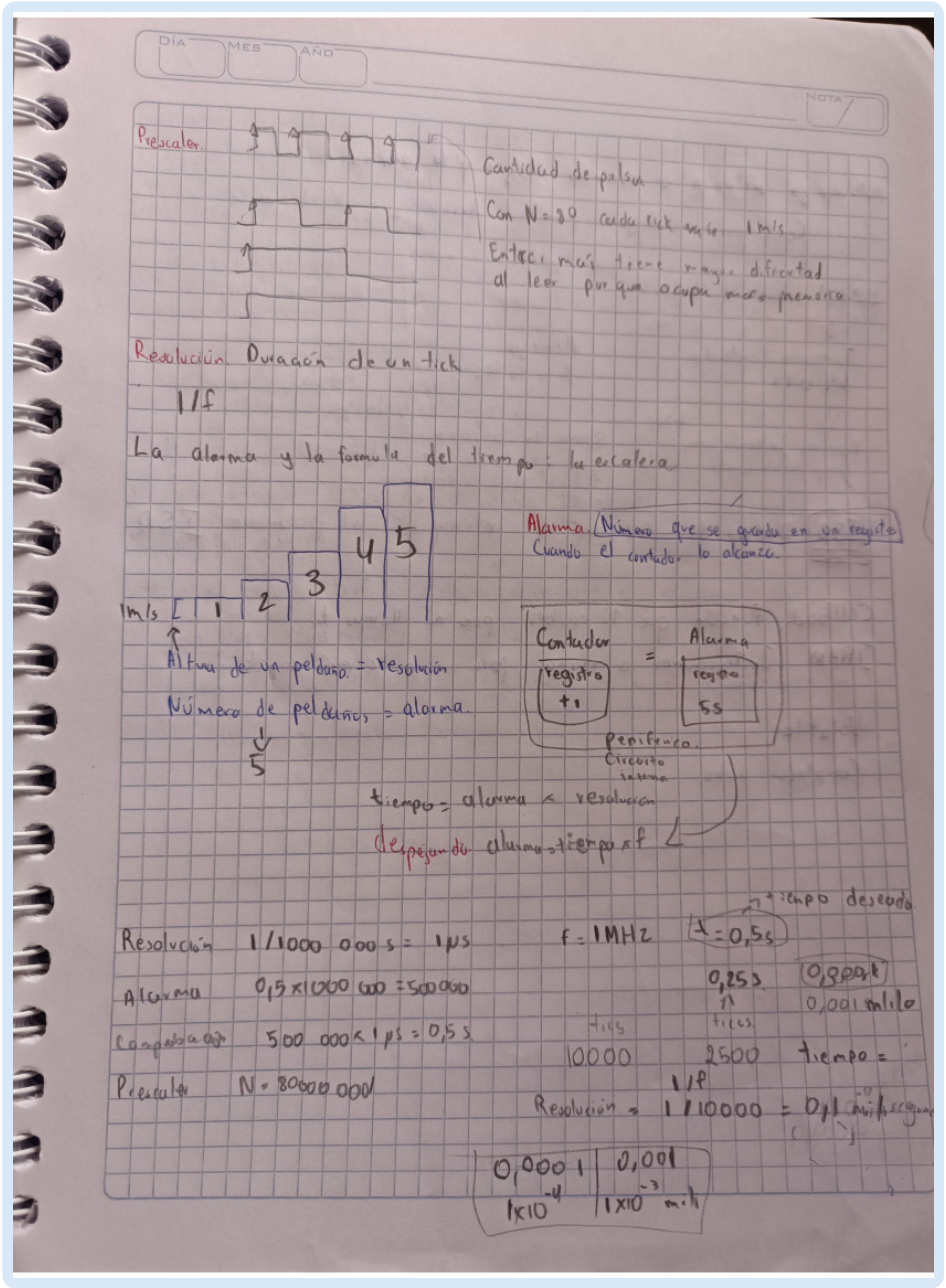
EN UNA IDEA

El contador es un registro del periférico, no una variable en RAM. Cuenta ticks sin una instrucción de CPU por cada incremento. 1/60 s ≈ 16,7 ms.

7.pdf · p. 1 · S3, pp. 7–15.

Prescaler, resolución y alarma

La escalera del tiempo



FÓRMULAS

f = reloj / N
resolución = 1 / f
tiempo = alarma / f
alarma = tiempo × f

EJEMPLO 1

80 MHz / 80 = 1 MHz.
1 tick = 1 µs.
500 000 ticks = 0,5 s.

EJEMPLO 2

10 kHz: 1 tick = 100 µs.
2500 ticks = 0,25 s.
0,0001 s = 0,1 ms, no 1 ms.

DESPEJAR f

f = 1 / resolución
f = alarma / tiempo
Usar segundos para que el resultado quede en Hz.
Guía completa: p. 20.

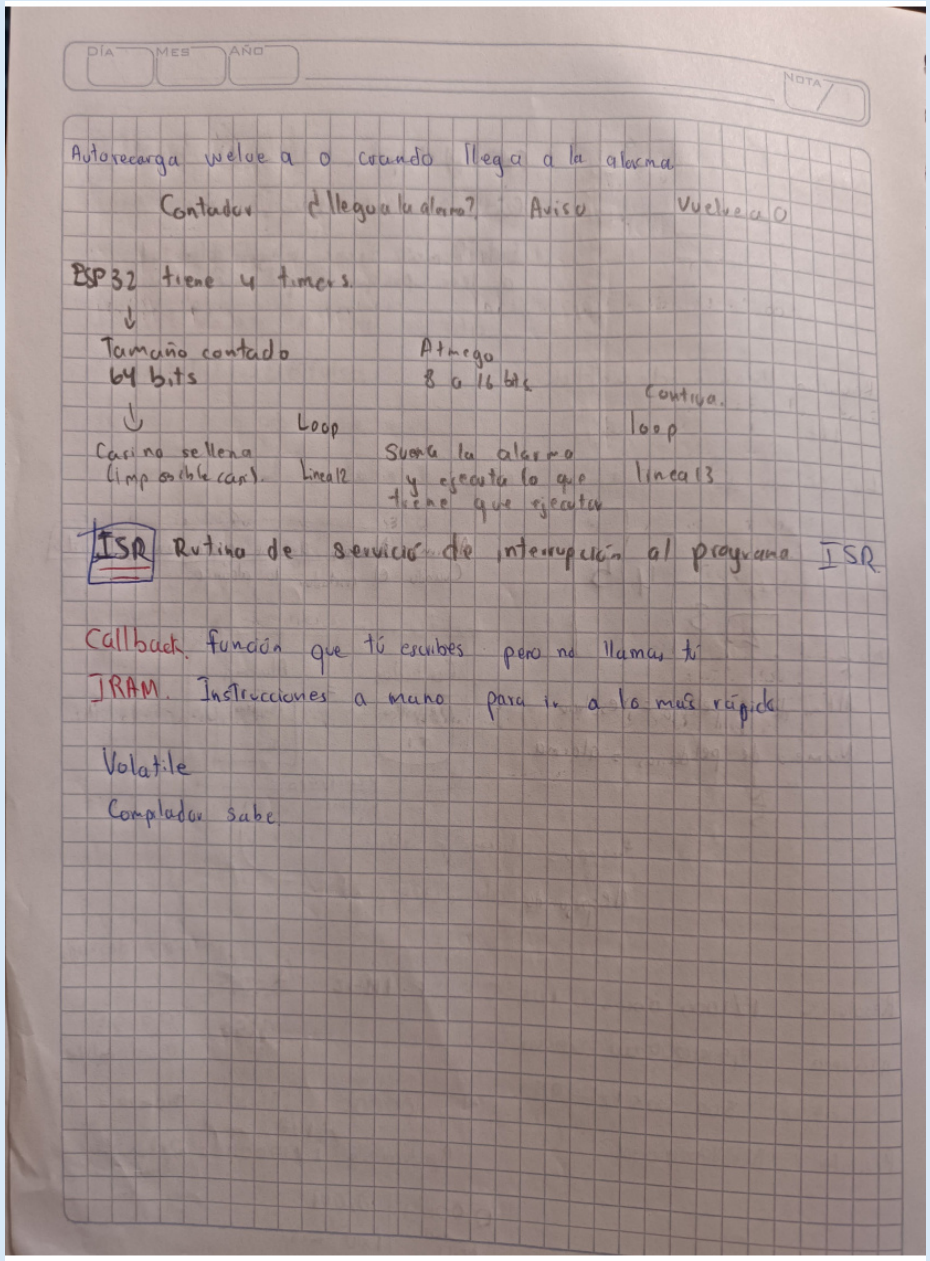
EN UNA IDEA

80 MHz ÷ 80 = 1 MHz: cada tick dura 1 µs. 500 000 ticks = 0,5 s. A 10 kHz: cada tick dura 100 µs; 2500 ticks = 0,25 s.

7.pdf · p. 2 · S3, pp. 16–21.

Autorrecarga e interrupciones

El temporizador avisa; la CPU atiende



AUTORRECARGA

El contador vuelve al valor configurado, aquí 0, al llegar a la alarma. No es lo mismo que desbordarse.

ISR Y CALLBACK

La ISR atiende la interrupción. Un callback es una función registrada para que otro componente la invoque.

VOLATILE

Indica posibles cambios fuera del flujo normal. No hace atómica una operación ni evita carreras.

AVISOS POR SEGUNDO

f_aviso = 1 / t = f / A.
A es el valor de alarma.
Si t = 0,25 s, hay 4 avisos por segundo. Recarga en 0.

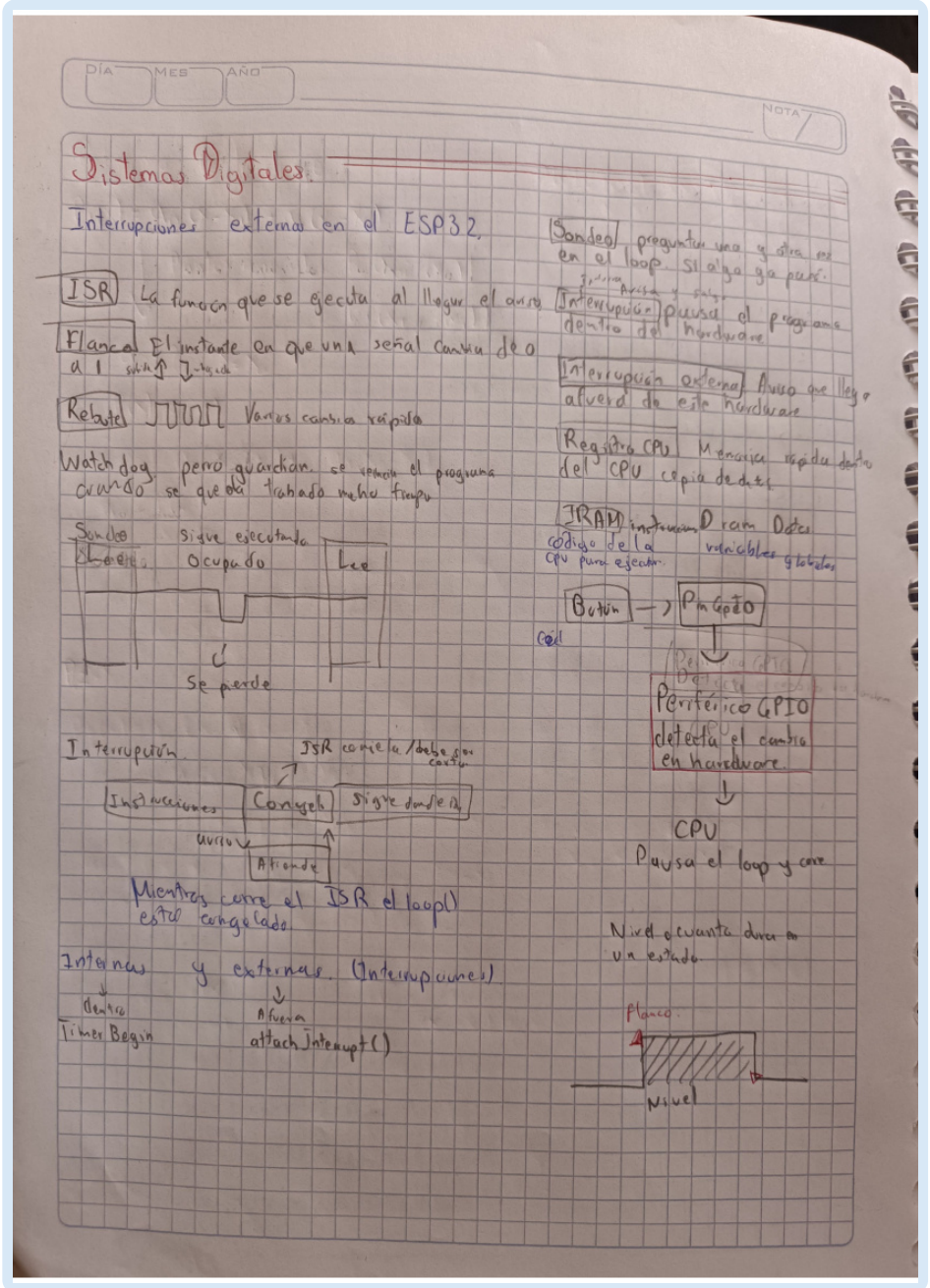
EN UNA IDEA

Autorrecarga no es desbordamiento. El ESP32 clásico tiene cuatro timers de 64 bits. volatile informa de posibles cambios externos, pero no garantiza atomicidad.

7.pdf · p. 3 · S3, pp. 23–28 y 35–40.

Interrupciones externas en el ESP32

Sondeo frente a interrupción



SONDEO

loop pregunta por el estado. Un pulso breve puede ocurrir entre dos lecturas y no ser detectado.

INTERRUPCIÓN

El GPIO detecta el cambio y solicita atención. Se ejecuta la ISR y se retoma la tarea interrumpida.

ISR CORTA

Dejar impresiones y esperas para loop. IRAM guarda instrucciones; Data RAM guarda datos.

CONFIGURACIÓN

Primero pinMode; después attachInterrupt, una vez en setup. Para pulsador con pull-up: FALLING al presionar.

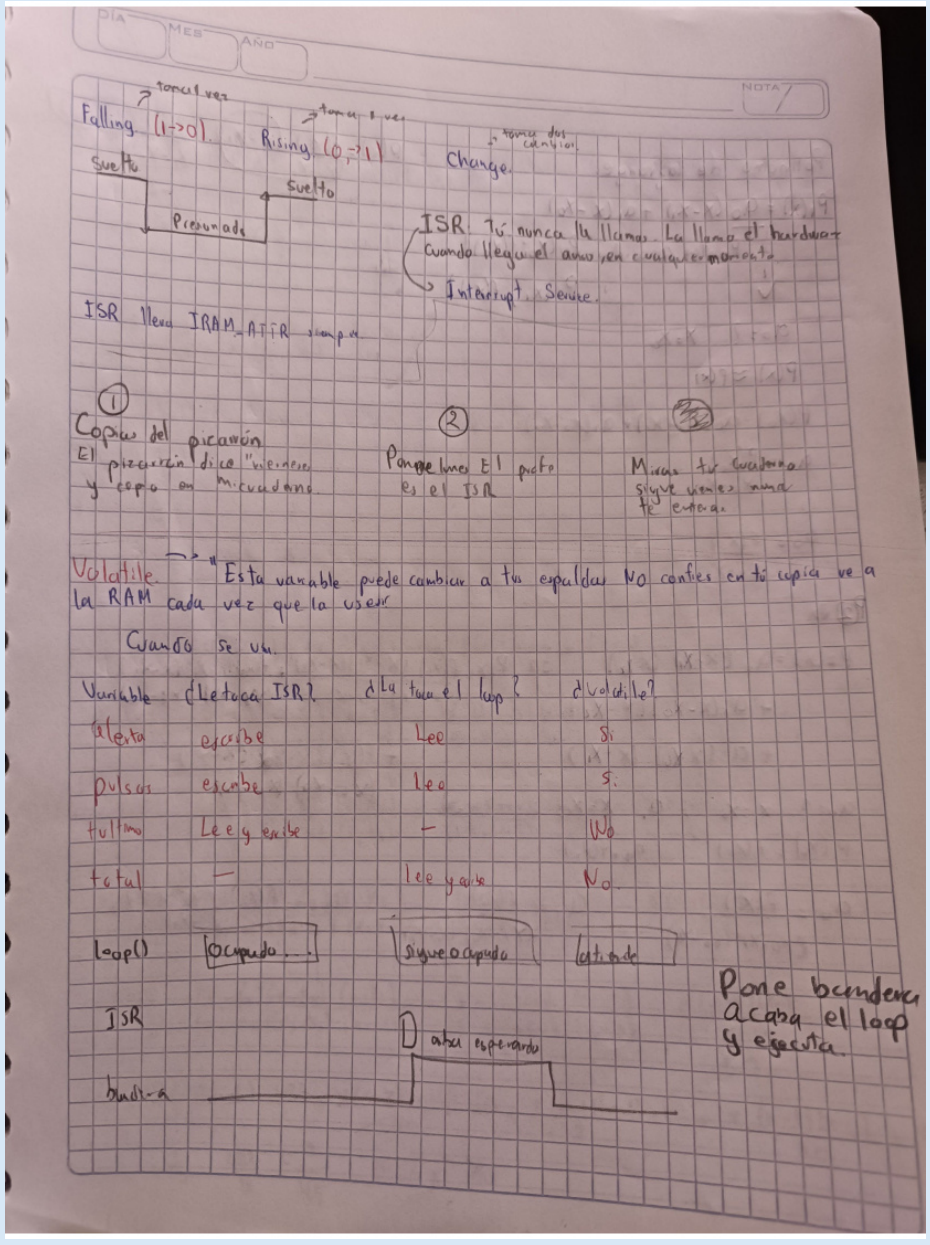
EN UNA IDEA

El periférico GPIO detecta el cambio; la CPU atiende la ISR y retoma la tarea. La ISR debe ser corta. IRAM guarda instrucciones; Data RAM guarda datos.

8.pdf · p. 1 · S4, pp. 6–13 y 23–31.

Flancos, volatile y patrón bandera

La ISR avisa; loop hace el trabajo



CON PULL-UP

FALLING: presionar.
RISING: soltar.
CHANGE: ambos flancos; el rebote puede generar más.

BANDERA

La ISR marca que ocurrió algo. loop revisa el aviso y realiza el trabajo pendiente.

LÍMITE

Un bool no cuenta eventos. volatile no sustituye la protección de accesos compartidos ni evita perder avisos concurrentes.

ANTIRREBOTE

El ejemplo de S4 ignora flancos a menos de 50 ms del último aceptado. Es un filtro temporal; no equivale a contar cada flanco.

EN UNA IDEA

Con pull-up: FALLING al presionar, RISING al soltar. Una bandera registra que pasó algo, no cuántas veces. volatile no reemplaza la protección de accesos compartidos.

8.pdf · p. 2 · S4, pp. 14–15 y 30–44.

Fórmulas para medir el tiempo

Basadas en S3, diapositivas 15–25. Los despejes permiten calcular la incógnita sin cambiar el modelo usado en clase.

01 · RELOJ Y PRESCALER

f = f_APB / N
N = f_APB / f
f_APB: reloj de entrada.
f: ticks por segundo (Hz).
N: divisor, sin unidades.
Ejemplo: 80 MHz / 80 = 1 MHz.

02 · FRECUENCIA Y PERÍODO

Δt = 1 / f
f = 1 / Δt
Δt: duración de un tick.
Para cualquier señal periódica:
T = 1 / f; f = 1 / T.
Ejemplo: 50 µs → 20 000 Hz.

03 · TIEMPO Y ALARMA

t = A × Δt = A / f
A = t × f
f = A / t
A: cantidad de ticks de alarma.
t: tiempo hasta el aviso.
Ejemplo: 250 000 / 1 MHz = 0,25 s.

04 · FRECUENCIA DEL AVISO

f_aviso = 1 / t = f / A
No confundir ticks/s con avisos/s.
Ejemplo: t = 0,25 s → 4 Hz.
Estas relaciones usan conteo desde 0 y, para repetición periódica, autorrecarga en 0.

05 · CONVERTIR ANTES DE CALCULAR

1 s = 1000 ms = 1 000 000 µs.
250 ms = 0,25 s; 50 µs = 0,00005 s.
1 kHz = 1000 Hz; 1 MHz = 1 000 000 Hz.
Si multiplicas segundos × ticks/segundo, obtienes ticks.

Límite del contador: con n bits, máximo = $2^n - 1$. Desde 0 y sin recarga, el desbordamiento llega tras 2^n ticks: $t_{desbordamiento} = 2^n / f$. Es distinto del aviso por alarma.

Fuente: S3_Temporizadores_ESP32.pdf · pp. 15–25. Se añaden despejes algebraicos de las fórmulas de clase.

De los datos a la respuesta

01 · UN AVISO CADA 250 ms

Datos: f = 1 000 000 ticks/s; t = 250 ms.
1. Convertir: 250 / 1000 = 0,25 s.
2. Aplicar: A = t × f = 0,25 × 1 000 000.
Resultado: A = 250 000 ticks.
Comprobación: 250 000 × 1 µs = 250 ms. Avisos: 1 / 0,25 = 4 por segundo.

02 · CALCULAR TIEMPO Y PRESCALER

Datos: f_APB = 80 MHz; f = 20 000 ticks/s; A = 5000.
1. Resolución: Δt = 1 / 20 000 = 0,00005 s = 50 µs.
2. Tiempo: t = 5000 / 20 000 = 0,25 s = 250 ms.
3. Divisor: N = 80 000 000 / 20 000 = 4000.
Resultado: aviso cada 0,25 s, con N = 4000.

03 · ENCONTRAR LA FRECUENCIA

Si un tick dura 50 µs: f = 1 / 0,00005 = 20 000 Hz.
Si A = 5000 y t = 0,25 s: f = A / t = 5000 / 0,25 = 20 000 Hz.
Ambos caminos coinciden. La frecuencia de aviso es 4 Hz, no 20 000 Hz.

Ejercicios de S3, p. 22:
A. f = 1 MHz y A = 2000 → t = 2 ms.
B. t = 10 ms y f = 1 MHz → A = 10 000 ticks.
C. f = 40 000 Hz y A = 40 000 → t = 1 s; N = 2000.

Ejemplos 1–2: captura de ejercicios aportada por el estudiante. Método: S3, pp. 16–22. Ejemplo 3: despejes de las mismas fórmulas.

TRANSPARENCIA ACADÉMICA

Declaración de uso de inteligencia artificial

Autoría de los apuntes

Los apuntes manuscritos incluidos en este resumen fueron elaborados por Richard Ariel Chamba Santin a partir de las clases de Sistemas Digitales. Las imágenes conservan su escritura y sus gráficos originales.

Apoyo utilizado

Se utilizó una herramienta de inteligencia artificial de OpenAI como apoyo para organizar el material, diseñar y maquetar el documento, mejorar su presentación y redactar notas de aclaración y complementos conceptuales.

Alcance de la asistencia

Las aclaraciones se prepararon tomando como base los apuntes y las diapositivas proporcionadas por el estudiante, con consulta complementaria de documentación técnica. Las notas añadidas se distinguen de las capturas manuscritas y no se presentan como parte de la escritura original.

Distinción de autoría

La asistencia de IA se declara expresamente como parte de la preparación de esta versión digital. No sustituye la autoría de los apuntes ni acredita por sí sola la comprensión de los contenidos por parte del estudiante.

Criterio de edición

Conservar el contenido original y sus ejemplos, corregir errores claros e identificar los añadidos. Los esquemas reconstruidos son didácticos. Las capturas de la versión visual conservan la escritura y los dibujos, incluidos los errores que se aclaran fuera de la imagen. No se han añadido fechas de clase ni datos personales no proporcionados.

RICHARD ARIEL CHAMBA SANTIN

Resumen de Sistemas Digitales · Noviembre de 2026